This article has been accepted for publication in IEEE Transactions on Wireless Communications. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TWC.2025.3619100

A Deep Reinforcement Learning with Transformer Integration for Directed Acyclic Graph Scheduling in Edge Networks

Xifei Song, Jie Feng, *Member, IEEE,* Lei Liu, *Member, IEEE,* Qingqi Pei, *Senior Member, IEEE,* F. Richard Yu, *Fellow, IEEE,* and Ning Zhang, *Senior Member, IEEE*

***Abstract*—The rapid adoption of 5G technology and Internet of things (IoT) devices has fueled significant growth in intelligent applications, increasing their complexity beyond simple task definitions. Scheduling intelligent applications modeled as directed acyclic graphs (DAGs) has thus emerged as a crucial challenge. Our proposed solution is a deep reinforcement learning (DRL) framework that uniquely integrates proximal policy optimization (PPO) with a transformer-based module for scheduling DAG applications. Unlike other approaches that rely on predefined priorities or static optimization algorithms, our approach enables agents to autonomously explore task execution orders and dynamically adapt to changing network resource conditions, learning optimal scheduling strategies. The algorithm leverages transformers to handle complex task dependencies, minimizing application duration and user energy consumption by jointly optimizing application processing order, task priorities, transmit power, offloading decisions, and computational frequency. Through a series of simulations, we prove the effectiveness of the proposed algorithm and demonstrate the performance comparison under different settings, providing a more flexible and robust solution for DAG scheduling in edge networks.**

***Index Terms*—Edge computing, scheduling, resource allocation, directed acyclic graph, deep reinforcement learning.**

I. Introduction

THE interplay between 5G technology and the Internet of things (IoT) has accelerated the evolution of intelligent services and infrastructure [1], [2], [3]. The high bandwidth and low latency of 5G enable the seamless interconnection among IoT devices, while the proliferation of IoT devices drives the continuous deployment and optimization of 5G networks. As a result, an increasing number of intelligent applications, including autonomous driving [4], [5], telemedicine [6], [7], augmented reality [8], [9], and virtual reality [10], [11], have emerged, driving an exponential surge in data generation. This rapid data growth places significant to traditional cloud-centric architectures. While cloud computing has been instrumental in offloading complex computations from resource-constrained IoT devices [12], offloading massive data to remote cloud servers introduces significant latency and network congestion, limiting real-time applications. To address latency-sensitive and resource-constrained scenarios, many applications leverage edge computing to bring computational capabilities closer to data sources [13]. However, compared to centralized cloud computing, edge environments are characterized by limited computational resources, dynamic network conditions, and energy constraints, making task scheduling a crucial factor in ensuring application performance.

Many intelligent applications consist of multiple interdependent tasks rather than standalone computations. These applications span various domains, including autonomous vehicles [14], scientific computing [15], autonomous robots [16], among others. They rely on continuous data processing across stages, where later stages depend on earlier results. Such applications can be naturally represented as directed acyclic graphs (DAGs), where nodes denote computational tasks and edges represent data dependencies. A representative example is the WATERS industrial challenge [17], which models an autonomous driving application as a complex DAG, encompassing interdependent tasks such as LiDAR grabber, localization, detection, path planner, and other workloads, with the primary goal of optimizing end-to-end response time (i.e., application duration) across the system. Scheduling these DAG-based applications in edge networks is challenging due to the constrained computational resources and limited energy supply of edge devices. A fundamental issue in DAG scheduling is the trade-off between computation offloading and communication overhead. Offloading tasks to remote servers alleviates local processing burdens but incurs transmission latency and additional energy consumption. Conversely, local execution minimizes transmission costs but may lead to prolonged execution time due to limited processing power.

For real-time applications such as autonomous vehicles and robots, high execution latency can significantly impact system performance and reliability [18], [19]. Numerous solutions have been proposed for DAG scheduling problem. For instance, the authors in [20] proposed a method to predict the minimum execution time of a DAG application under given resource constraints by solving linear programming (LP) and mixed-integer linear programming (MILP) models. Baruah [21] explored the use of Integer Linear Programming (ILP) to represent and solve DAG application scheduling

Xifei Song, Jie Feng, and Qingqi Pei are with the State Key Laboratory of ISN and the Shaanxi Key Laboratory of Blockchain and Secure Computing, School of Telecommunications Engineering, Xidian University, Xi'an, Shaanxi 710071, China (e-mail: 20011110025@stu.xidian.edu.cn; jiefengcl@163.com; qqpei@mail.xidian.edu.cn).

Lei Liu is with the Guangzhou Institute of Technology, Xidian University, Guangzhou 510555, China (e-mail: tianjiaoliulei@163.com).

F. Richard Y is with the Department of Systems and Computer Engineering, Carleton University, Ottawa, ON K1S 5B6, Canada (e-mail: richard.yu@carleton.ca).

Ning Zhang is with the Department of Electrical and Computer Engineering, University of Windsor, Windsor, ON N9B 3P4, Canada (e-mail: ning.zhang@uwindsor.ca).

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on November 05,2025 at 08:36:05 UTC from IEEE Xplore. Restrictions apply.
© 2025 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Wireless Communications. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TWC.2025.3619100

problems on multiprocessor platforms, aiming to optimize task execution order and minimize end-to-end latency. While these approaches have their merits, they often face challenges in dynamic edge networks. First, these approaches typically assume a relatively static system state, which does not align with the dynamic nature of edge computing, characterized by fluctuating channel conditions and varying node availability. Second, they often ignore the randomness of task arrivals, a common characteristic in real-world scenarios. Therefore, it is essential to develop adaptive scheduling strategies to optimize DAG-based application execution in edge networks. Furthermore, energy consumption has become a pivotal consideration in edge networks, especially for battery-powered devices or power-constrained nodes. Many scheduling algorithms primarily focus on optimizing execution time while overlooking energy consumption. However, as IoT and mobile devices become increasingly prevalent, it is necessary to design scheduling strategies that jointly optimize application duration and energy consumption.

To address the challenges of DAG scheduling in edge networks, we introduce a novel DRL framework to optimize application duration and reduce user energy consumption. The primary contributions can be summarized as follows:

- We propose a DAG application scheduling framework designed for multi-user scenarios. Unlike methods that determine task priorities and placement strategies in stages, this framework simultaneously assigns task priorities and placement strategies based on environmental information, enhancing flexibility and optimization potential.
- We propose an optimization problem to minimize the average application duration and user energy consumption, achieved by jointly optimizing application processing order, task priorities, transmit power, offloading decisions, and computational frequency.
- We have developed a proximal policy optimization (PPO) algorithm that integrates a transformer module for extracting DAG feature information. The depth-based position encoding enhances the agent's ability to understand information. The policy network is also hierarchically designed to reduce the agent's action space.
- We demonstrated that the algorithm exhibits good convergence properties through extensive simulation experiments. In addition, we validated the effectiveness of the proposed scheme from multiple perspectives, including reinforcement learning algorithms, neural network models, task prioritization, and placement strategies. Notably, the proposed scheme outperforms other algorithms.

The remainder of this paper is structured as follows. Section II reviews related work. Section III describes the system, application, and scheduling models used. Section IV formulates the problem we aim to address. Section V details the implementation specifics of our DRL-based DAG scheduling algorithm. Section VI presents our experimental settings and evaluation metrics. Section VII concludes the paper.

II. Related Work

The scheduling of DAG-based application has been extensively studied. This section reviews key contributions across different domains, highlighting their strengths and limitations in addressing DAG scheduling challenges.

Some researchers have employed mathematical programming methods, such as mixed-integer programming [20], [22] and dynamic programming [23], [24], to solve scheduling problems. Hu *et al.* [25] proposed a dynamic programming algorithm for scheduling static applications in vehicular edge computing systems, integrating the common mixed-criticality mode-switch scheme to ensure that applications are served according to their criticality levels. Khaleel [26] introduced a hybrid integer multi-objective decision-making framework, augmented with a gravity reference point technique, designed to optimize the computation offloading of DAG applications across cloud and edge servers. These methods excel in handling small-scale, static problems, providing theoretical optimal solutions.

Other researchers have employed heuristic or meta-heuristic algorithms to optimize their strategies [27], [28], [29]. Heuristic algorithms are typically designed for specific problems, relying on predefined rules or strategies, and can quickly generate solutions for large-scale problems. Zhang *et al.* [30] proposed an enhanced priority-based heuristic that considers DAG structure attributes, task execution times, and data transmission time estimates during task prioritization to minimize the batched makespan. However, these algorithms often need more global optimization capabilities, resulting in suboptimal solutions that may not guarantee performance close to the global optimum, especially in complex and dynamic scenarios. In contrast, meta-heuristic algorithms, such as genetic algorithms and particle swarm optimization, perform well in exploring the solution space and exhibit good global optimization capabilities. Yang *et al.* [31] introduced a joint computation offloading and resource allocation algorithm for DAG tasks in MEC-enabled IoV networks, using an adaptive hybrid particle swarm with genetic algorithm to optimize offloading strategies, resource allocation, and load balancing. However, these algorithms are also challenging to implement in edge environments. The reason is that these meta-heuristic algorithms typically require a large number of iterations to converge, which cannot meet the real-time response requirements.

Given the limitations of traditional methods, researchers have started to explore more adaptive and flexible scheduling strategies. Deep reinforcement learning (DRL) methods have shown significant potential in solving complex scheduling problems, particularly in dynamic and uncertain environments [32]. For instance, the authors in [33] demonstrated the advantages of DRL in handling highly dynamic task arrival and time-varying system states. Furthermore, Mnih *et al.* [34] showed that agents can learn control policies from high-dimensional sensory inputs, proving DRL particularly suitable for handling environments with high uncertainty and complex state spaces. These characteristics make DRL ideal for tackling complex scheduling problems in edge environments. However, existing DRL methods for DAG scheduling still often rely on traditional techniques such as breadth first search [35], [36], [37], critical path analysis [38], [39], or task priority calculation [40], [41] to determine the priority of tasks. While this approach simplifies the problem, it limits the model's

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on November 05,2025 at 08:36:05 UTC from IEEE Xplore. Restrictions apply.
© 2025 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Wireless Communications. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TWC.2025.3619100

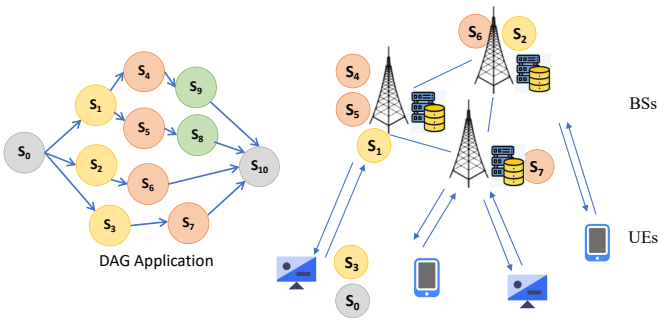


Fig. 1. System model.

ability to explore and learn optimal strategies thoroughly.

III. System Scenario

In the below content, we present the system model and application model respectively, followed by a detailed description of the scheduler's processing logic for user requests in edge computing system.

A. System Model

Fig. 1 illustrates an overview of our system. In the considered system, there are many applications with internal dependencies continuously generated by N users. Let $\mathcal{N} = \{1, 2, \cdots, N\}$ denote the set of users. The arrivals of users and their applications follow a Poisson process, meaning that the inter-arrival times are exponentially distributed. Due to the inherent limitations of user equipment computing power and battery life, these applications need to be offloaded to edge servers for execution via wireless communications. We assume that the system has M base stations (BSs). Each BS is co-located with an edge server, enabling low-latency computing services at the network edge. The BSs are interconnected via a high-bandwidth fiber optic link. Consistent with [42], we assume the system utilizes a universal frequency reuse scheme. Under this scheme, all BSs can leverage the same radio resources, promoting efficient spectrum utilization. Users within each BS's coverage area employ orthogonal frequency-division multiple access (OFDMA) for data transmission. This technique allows multiple devices to concurrently transmit data using non-overlapping frequency channels, minimizing interference and maximizing network capacity.

Our system provides flexibility in task execution, allowing applications to be processed on the user's local device or an edge server. For clarity, we introduce the set $\mathcal{S} = s_0 \cup \mathcal{S}' = \{s_0, s_1, \cdots, s_M\}$, where s_0 denotes the user device initiating the tasks and $\mathcal{S}'$ denotes the set of server devices. Specifically, the devices $s_1 \sim s_M$ denotes the M edge servers.

B. Application Model

The application generated by user n is a collection of dependent atomic tasks that can be offloaded to a set of edge servers. In this article, we represent the application as a DAG. Let $G_n = (V_n, E_n)$ denote the application n, where $V_n = \{v_{n,1}, v_{n,2}, \cdots, v_{n,J}\}$ is the set of vertexes, in

TABLE I
Notation Definitions

Symbol	Definition
$G_n = \{V_n, E_n\}$	DAG of an application, where V_n is the task set and E_n denotes data dependencies.
$\mathcal{G}_n = \{\mathcal{V}_n, \mathcal{E}_n\}$	Extended DAG with virtual tasks, where $\mathcal{V}_n$ and $\mathcal{E}_n$ denote the updated task set and dependencies.
$v^w_{n,j}$	CPU cycles required by task $v_{n,j}$
$e^w_{n,i,j}$	Data size from task $v_{n,i}$ to task $v_{n,j}$.
$S = \{s_0\} \cup S'$	s_0 represents the user device initiating the tasks, and S' denotes the set of server devices.
$x^q_{\mathcal{G}_n}$	Binary variable indicating if application $\mathcal{G}_n$ is scheduled in the event q.
$x^o_{v_{n,j}}$	Binary variable indicating if task $v_{n,j}$ is scheduled at position o.
$x^{s_b}_{v_{n,j}}$	Binary variable indicating if task $v_{n,j}$ is offloaded to s_b.
$p^{v_{n,j}}_{n,m}$	Transmission power allocated form device n to BS m for task $v_{n,j}$.
$f^{s_b}_{v_{n,j}}$	Computing resources allocated to task $v_{n,j}$ on device s_b.
P_{max}	Maximum allowable transmit power.
$F^{s_b}_{cap}$	Computing resource capacity of device s_b.
$\texttt{Pre}(\cdot)$, $\texttt{Suc}(\cdot)$	Functions that return the predecessor and successor nodes of a task.
$\texttt{CP}(\cdot)$	Function that returns the set of critical tasks in a DAG.
U_m	Number of users associated with BS.
$r_{n,m}$ $(r^{ul}_{a,b})$	Transmit rate for uplink data from user n to BS m.
$r_{m,n}$ $(r^{dl}_{a,b})$	Transmit rate for downlink data from BS m to user n.
depth$(\cdot)$	Function that returns the depth of a task in the DAG.

which $v_{n,i}$ depicts the i-th task of application n. In addition, $E_n = \{e_{n,i,j} = (v_{n,i}, v_{n,j}) \mid v_{n,i}, v_{n,j} \in V_n, i \neq j\}$ is the set of directed edges, where $e_{n,i,j}$ illustrates the data flow between task $v_{n,i}$ and task $v_{n,j}$. Specifically, task $v_{n,j}$ relies on the output of task $v_{n,i}$ as input. We can say that task $v_{n,j}$ is a successor of task $v_{n,i}$ and task $v_{n,i}$ is a predecessor of task $v_{n,j}$. It is a remarkable fact that a task within complex applications can have multiple predecessors and successors. Therefore, we design two functions $\texttt{Pre}(\cdot)$ and $\texttt{Suc}(\cdot)$ to extract a task's set of predecessors and successors, respectively (i.e. $v_{n,i} \in \texttt{Pre}(v_{n,j})$ and $v_{n,j} \in \texttt{Suc}(v_{n,i})$).

To ensure the successful execution of the application, the initiator, i.e., the user equipment, must transfer the necessary input data to the corresponding tasks on different edge devices. Once all tasks are completed, the calculation results must be returned to the user equipment for completion. For clarity, we augment each application's DAG topology with a virtual entry task $v_{n,0}$ and a virtual exit task $v_{n,J+1}$. These two virtual tasks must be placed on the user equipment and connected to other tasks through directed edges. Therefore, we extend the DAG representation of application n to $\mathcal{G}_n = (\mathcal{V}_n, \mathcal{E}_n)$, where $\mathcal{V}_n$ is the set of all vertices, representing the set of computing tasks and virtual tasks, and $\mathcal{E}_n$ is the set of corresponding directed edges, which represent the specific data flow direction during the execution of the application. Furthermore, we associate a weight $v^w_{n,i}$ with each vertex $v_{n,i}$ in the DAG $\mathcal{G}_n$. This weight represents the number of CPU cycles required to process the corresponding task $v_{n,i}$. Notably, the virtual entry task $v_{n,0}$

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on November 05,2025 at 08:36:05 UTC from IEEE Xplore. Restrictions apply.
© 2025 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Wireless Communications. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TWC.2025.3619100

and virtual exit task $v_{n,J+1}$ have zero workload, meaning $v^w_{n,0} = v^w_{n,J+1} = 0$. Additionally, each directed edge $e_{n,i,j}$ in the DAG $\mathcal{G}_n$ is assigned a weight $e^w_{n,i,j}$. This weight signifies the data transfer from task $v_{n,i}$ to task $v_{n,j}$.

C. Scheduling Framework

For each arriving user request, the scheduler must dynamically determine scheduling strategies for all tasks in application $\mathcal{G}_n$ based on application characteristic information, internal dependencies, and network status. The prerequisite for making scheduling decisions is determining which tasks are ready. For a task $v_{n,i}$, it can only be considered schedulable if all its predecessors have been completed, denoted by $\texttt{Pre}(v_{n,i})$. Due to the structural characteristics of DAGs, multiple schedulable tasks may exist simultaneously when executing $\mathcal{G}_n$. Furthermore, as new applications arrive during execution, the scheduler's queue will continue to grow, with an increasing number of tasks waiting to be scheduled. Since system resources are limited and the number of tasks and processing progress within each application varies, the scheduler's decision on task execution order and task placement strategy significantly impacts the completion time of applications.

First, we define task prioritization in a multi-user scenario, covering both the execution order of applications and the priorities of their internal tasks. Once the application $\mathcal{G}_n$ is selected during the q-th schedule event, $x^q_{\mathcal{G}_n} = 1$. The one-hot vector $\mathbf{x}^q_{\mathcal{G}} = \left[x^q_{\mathcal{G}_0}, x^q_{\mathcal{G}_1}, \cdots, x^q_{\mathcal{G}_n}\right]^\top$ indicates the applications selected by the scheduler during the q-th scheduling event. Scheduling events of the entire scheduling process are represented by set $\mathcal{Q}$. The matrix $\mathbf{X}^{\mathcal{Q}}_{\mathcal{G}} = \left[\mathbf{x}^0_{\mathcal{G}}, \mathbf{x}^1_{\mathcal{G}}, \cdots, \mathbf{x}^Q_{\mathcal{G}}\right]$ represents the application execution sequence throughout the entire scheduling process. The application $\mathcal{G}_n$ consists of $(J+2)$ tasks, where $x^o_{v_{n,j}} = 1$ indicates that task $v_{n,j}$ is scheduled in the o-th position. Thus, the one-hot vector $\mathbf{x}^{\mathcal{O}}_{v_{n,j}}$ represents the priority of task $v_{n,i}$. Therefore, the matrix $\mathbf{X}^{\mathcal{O}}_{\mathcal{V}}$ reflects the scheduling order of all tasks in application $\mathcal{G}_n$.

Next, we define the scheduler's task placement process, which includes decisions on task offloading and resource allocation. We also define a binary variable $x^{s_b}_{v_{n,j}}$ indicates whether task $v_{n,j}$ is offloaded to server s_b, and the $(m+1)$-dimensional one-hot vector $\mathbf{x}^{\mathcal{S}}_{v_{n,j}}$ represents the placement strategies of $v_{n,j}$ on all devices in $\mathcal{S}$. Notably, if $v_{n,j}$ is executed on a server while its predecessor node remains on a local device, wireless transmission is required. The variable $P^{v_{n,j}}_{n,m}$ is the transmission power allocated to task $v_{n,j}$. Similarly, the variable $f^{s_b}_{v_{n,j}}$ denotes the computing resources allocated to task $v_{n,j}$ by device s_b, and the vector $\mathbf{f}^S_{v_{n,j}}$ captures the computing resources obtained by $v_{n,i}$ across the device set $\mathcal{S}$. Consequently, the matrix $\mathbf{X}^{\mathcal{S}}_{\mathcal{V}}$ represents the offloading strategy for application $\mathcal{G}_n$, the vector $P_{\mathcal{V}}$ denotes the transmission power assigned to $\mathcal{G}_n$, and the matrix $\mathbf{F}^{\mathcal{S}}_{\mathcal{V}}$ specifies the computing resources allocated to $\mathcal{G}_n$.

IV. Problem Description

In this section, we delve into the cost analysis of task scheduling in DAG applications, focusing on two primary metrics: application duration and energy consumption.

A. Application Duration

We define the application duration as the elapsed time from when the user initiates the application request to when the corresponding result is returned to the user. In the improved DAG representation $\mathcal{G}_n$, a virtual exit node is introduced on the user device to capture the computational results of the application. This permits us to focus solely on the communication delay for the task to get the necessary input and the computational delay to complete the task.

1) Communication delay: As mentioned, user equipment and edge servers exchange data to facilitate collaborative application execution. Assuming task $v_{n,i}$ resides on device s_a and task $v_{n,j}$ resides on s_b. The rate at which task $v_{n,i}$ transmits $e^w_{n,i,j}$ bits of data to task $v_{n,j}$ depends on the specific transmission path.

Uplink Transmission: The transmit rate for uplink data from user $n \in \mathcal{N}$ to BS $m \in \mathcal{M}$ can be calculated as follows:

$$r^{ul}_{a,b} = r_{n,m} = \frac{W}{U_m}\log_2\left(1 + \frac{P^{v_{n,j}}_{n,m} g_{n,m}}{\sum\limits_{p,q\neq n,m} P_{max} g_{p,q} + \sigma^2}\right), \tag{1}$$

where W represents the total bandwidth of the system. Through frequency reuse, multiple BSs can utilize the same radio resources. U_m signifies the number of users associated with BS m, meaning each user can access $1/U_m$ of the total bandwidth. $P^{v_{n,j}}_{n,m}$ denotes the transmit power for task $v_{n,j}$ from the user n to the BS m, with the maximum allowable transmit power P_{max}. $g_{n,m}$ is the channel gain from the user n to BS m, and σ^2 is the Gaussian noise power.

Downlink Transmission: The transmit rate for downlink data from BS $m \in \mathcal{M}$ to user $n \in \mathcal{N}$ can be calculated as follows:

$$r^{dl}_{a,b} = r_{m,n} = \frac{W}{U_m}\log_2\left(1 + \frac{P_{m,n} g_{m,n}}{\sum\limits_{p,q\neq m,n} P_{max} g_{p,q} + \sigma^2}\right), \tag{2}$$

where $P_{m,n}$ denotes transmit power of BS m to user n.

Optical fiber transmission: In scenarios where $v_{n,i}$ and $v_{n,j}$ are assigned to different edge server, device s_a and s_b transmit data directly over optical fiber wired link. At this point, $r_{a,b} = c_f$.

In summary, the transmission time required from task $v_{n,i}$ to task $v_{n,j}$, corresponding to edge $e_{n,i,j}$, is as follows:

$$\begin{aligned}\tau^{tra}_{e_{n,i,j}} &= \tau^{tra}_{v_{n,i}\to v_{n,j}} \\ &= \begin{cases} 0, & \text{if } s_a = s_b, \\ \dfrac{e^w_{n,i,j}}{r^{ul}_{a,b}} \times x^{s_a}_{v_{n,i}} x^{s_b}_{v_{n,j}}, & \text{if } s_a \in \mathcal{N},\ s_b \in \mathcal{M}, \\ \dfrac{e^w_{n,i,j}}{r^{dl}_{a,b}} \times x^{s_a}_{v_{n,i}} x^{s_b}_{v_{n,j}}, & \text{if } s_a \in \mathcal{M},\ s_b \in \mathcal{N}, \\ \dfrac{e^w_{n,i,j}}{c_f} \times x^{s_a}_{v_{n,i}} x^{s_b}_{v_{n,j}}, & \text{otherwise,} \end{cases}\end{aligned} \tag{3}$$

where $e^w_{n,i,j}$ represents the amount of data transmitted from $v_{n,i}$ to $v_{n,j}$.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on November 05,2025 at 08:36:05 UTC from IEEE Xplore. Restrictions apply.
© 2025 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Wireless Communications. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TWC.2025.3619100

Before executing the task $v_{n,j}$, the task must receive the necessary data from all its predecessors $v_{n,i} \in \texttt{Pre}(v_{n,j})$. The time required to receive these input data is:

$$\tau_{v_{n,j}}^{prep} = \max \tau_{e_{n,i,j}}^{tra} = \max \tau_{v_{n,i} \to v_{n,j}}^{tra}, \forall v_{n,i} \in \texttt{Pre}(v_{n,j}). \tag{4}$$

2) Computation delay: Once the data preparation for task $v_{n,j}$ is complete, the task is ready for execution. The decision to execute task $v_{n,j}$ on device s_b is indicated by the binary variable $x_{v_{n,j}}^{s_b}$. Specifically, if $x_{v_{n,j}}^{s_b} = 1$, task $v_{n,j}$ is assigned to device s_b, and subsequently, server s_b allocates the necessary computational resources, denoted by $f_{v_{n,j}}^{s_b}$, to execute the task. The total execution time of task $v_{n,j}$ on device s_b can be calculated as follows:

$$\tau_{v_{n,j}}^{exec} = \frac{v_{n,j}^{w}}{f_{v_{n,j}}^{s_b}} \times x_{v_{n,j}}^{s_b}, \tag{5}$$

where $v_{n,j}^{w}$ represents the number of CPU cycles required to process task $v_{n,j}$. Device s_b possesses a total computational resource capacity denoted by $F_{cap}^{s_b}$. The utilization of this resource is subject to the following constraint:

$$\sum_{n \in \mathcal{N}} \sum_{v_{n,j} \in \mathcal{V}_n} f_{v_{n,j}}^{s_b} \times x_{v_{n,j}}^{s_b} \leq F_{cap}^{s_b}. \tag{6}$$

The duration of task $v_{n,j}$ on device s_b can be calculated based on the preparation time and execution time of task $v_{n,j}$, which can be given by:

$$\tau_{v_{n,j}} = \tau_{v_{n,j}}^{prep} + \tau_{v_{n,j}}^{exec}. \tag{7}$$

Due to the inherent topology characteristics of $\mathcal{G}_n$, the application exhibits a large number of parallel tasks during execution. Consequently, the overall duration of the application $\mathcal{G}_n$, denoted by $T_{\mathcal{G}_n}$, can be calculated as follows:

$$T_{\mathcal{G}_n} = \sum_{v_{n,j} \in \texttt{CP}(\mathcal{G}_n)} \tau_{v_{n,j}} = \sum_{v_{n,j} \in \texttt{CP}(\mathcal{G}_n)} \left(\tau_{v_{n,j}}^{prep} + \tau_{v_{n,j}}^{exce} \right), \tag{8}$$

where the function $\texttt{CP}(\cdot)$ is employed to retrieve the critical path task set within $\mathcal{G}_n$.

B. Energy Consumption

In this study, our primary focus is minimizing user equipment energy consumption while executing applications. This emphasis is driven by the fact that edge servers are typically connected to a stable power supply, making their energy consumption less critical for optimization. From the perspective of user n, task execution's energy consumption can be divided into two main parts: data transmission and task execution.

For the data link $e_{n,i,j}$, when task $v_{n,i}$ is located on device n (i.e., $x_{v_{n,i}}^{s_0} = 1$) and task $v_{n,j}$ is located on another device, the transmission time and transmission energy consumption of device n are calculated as follows:

$$T_{\mathcal{G}_n}^{tra} = \sum_{e_{n,i,j} \in \mathcal{E}} \left[\tau_{e_{n,i,j}}^{tra} \times x_{v_{n,i}}^{s_0} \times (1 - x_{v_{n,j}}^{s_0}) \right], \tag{9}$$

$$\begin{aligned} E_{\mathcal{G}_n}^{tra} &= P_{n,m}^{v_{n,j}} T_{\mathcal{G}_n}^{tra} \\ &= \sum_{e_{n,i,j} \in \mathcal{E}} \left[P_{n,m}^{v_{n,j}} \tau_{e_{n,i,j}}^{tra} \times x_{v_{n,i}}^{s_0} \times (1 - x_{v_{n,j}}^{s_0}) \right]. \end{aligned} \tag{10}$$

Similarly, the total execution energy consumption of device n during the processing of application $\mathcal{G}_n$ is:

$$\begin{aligned} E_{\mathcal{G}_n}^{exec} &= P_n^{exec} T_{\mathcal{G}_n}^{exec} = \sum_{v_{n,j} \in \mathcal{V}} P_n^{exec} \left[\tau_{v_{n,j}}^{exec} \times x_{v_{n,j}}^{s_0} \right], \\ P_n^{exec} &= \zeta_n f_{v_{n,j}}^3, \end{aligned} \tag{11}$$

where P_n^{exec} is the execution power of local device n when processing the task. ζ_n denotes the effective switched capacitance of device n, and $f_{v_{n,j}}$ represents the CPU frequency during execution.

Therefore, the total energy consumption of local device n for executing application $\mathcal{G}_n$ can be expressed as:

$$E_{\mathcal{G}_n} = E_{\mathcal{G}_n}^{tra} + E_{\mathcal{G}_n}^{exec} + P_n^{idle} \left(T_{\mathcal{G}_n} - T_{\mathcal{G}_n}^{tra} - T_{\mathcal{G}_n}^{exec} \right). \tag{12}$$

C. Problem Formulation

Our goal is to design an algorithm that determines the optimal placement strategy for each user's application in the edge computing system to minimize the average consumption of all users. Specifically, we aim to minimize both the average application duration and the energy consumption of users, as reflected in the following optimization objective:

$$\Omega = \frac{1}{N} \sum_{n \in \mathcal{N}} \left(\alpha T_{\mathcal{G}_n} + \beta E_{\mathcal{G}_n} \right), \tag{13}$$

where $T_{\mathcal{G}_n}$ and $E_{\mathcal{G}_n}$ represent the application duration and energy consumption of the application n, respectively, which are calculated by (8) and (12). The parameters α and β balance the trade-off between application duration and energy consumption.

Ultimately, we formulate an optimization problem to minimize the average weighted cost of users in the system, jointly optimizing application processing order, task priorities, offloading decisions, transmit power, and computational frequency. The joint optimization problem is formulated as:

$$\min_{\mathbf{X}_{\mathcal{G}}^{\mathcal{Q}}, \mathbf{X}_{\mathcal{V}}^{\mathcal{O}}, \mathbf{X}_{\mathcal{V}}^{\mathcal{S}}, \mathbf{P}_{\mathcal{V}}, \mathbf{F}_{\mathcal{V}}^{\mathcal{S}}} \Omega = \frac{1}{N} \sum_{q \in \mathcal{Q}} \sum_{n \in \mathcal{N}} \left(\alpha T_{\mathcal{G}_n} + \beta E_{\mathcal{G}_n} \right) \times x_{\mathcal{G}_n}^{q}$$

s.t.

$$\begin{aligned}
&(\text{C1}): \ \alpha + \beta = 1, \\
&(\text{C2}): \ \mathbf{X}_{\mathcal{G}}^{\mathcal{Q}} = [x_{\mathcal{G}_n}^{q}]_{|\mathcal{N}| \times |\mathcal{Q}|}, \ x_{\mathcal{G}_n}^{q} = \{0, 1\}, \\
&(\text{C3}): \ \sum_{n \in \mathcal{N}} x_{\mathcal{G}_n}^{q} \leq 1, \\
&(\text{C4}): \ \mathbf{X}_{\mathcal{V}}^{\mathcal{O}} = [x_{v_{n,j}}^{o}]_{|\mathcal{V}_n| \times |\mathcal{V}_n|}, \ x_{v_{n,j}}^{o} = \{0, 1\}, \\
&(\text{C5}): \ \sum_{o=0}^{|\mathcal{V}_n|} x_{v_{n,j}}^{o} = 1, \ \sum_{v_{n,j} \in \mathcal{V}_n} x_{v_{n,j}}^{o} = 1, \\
&(\text{C6}): \ \mathbf{X}_{\mathcal{V}}^{\mathcal{S}} = [x_{v_{n,j}}^{s_b}]_{|\mathcal{S}| \times |\mathcal{V}_n|}, \ x_{v_{n,j}}^{s_b} = \{0, 1\}, \\
&(\text{C7}): \ \sum_{s_b \in \mathcal{S}} x_{v_{n,j}}^{s_b} = 1, \\
&(\text{C8}): \ \mathbf{P}_{\mathcal{V}} = \left[P_{n,m}^{v_{n,j}} \right]_{1 \times |\mathcal{V}_n|}, \ P_{min} \leq P_{n,m}^{v_{n,j}} \leq P_{max}, \\
&(\text{C9}): \ \mathbf{F}_{\mathcal{V}}^{\mathcal{S}} = \{f_{v_{n,j}}^{s_b}\}_{|\mathcal{S}| \times |\mathcal{V}_n|}, \ f_{min} \leq f_{v_{n,j}}^{s_b} \leq F_{cap}^{s_b}, \\
&(\text{C10}): \ \sum_{n \in \mathcal{N}} \sum_{v_{n,j} \in \mathcal{V}_n} f_{v_{n,j}}^{s_b} \times x_{v_{n,j}}^{s_b} \leq F_{cap}^{s_b}.
\end{aligned} \tag{14}$$

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on November 05,2025 at 08:36:05 UTC from IEEE Xplore. Restrictions apply.
© 2025 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Wireless Communications. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TWC.2025.3619100

Constraint (C1) ensures that the coefficients α and β sum up to 1, thus balancing the trade-off between application duration and energy consumption. Constraint (C2) defines the execution state matrix $\mathbf{X}_{\mathcal{G}}^{\mathcal{Q}}$ of the scheduler, where $x_{\mathcal{G}_n}^{q}$ indicates whether application $\mathcal{G}_n$ is executed by the scheduler during the q-th scheduling event. Constraint (C3) states that the scheduler selects at most one application during the q-th scheduling event. Constraint (C4) specifies the priority matrix $\mathbf{X}_{\mathcal{V}}^{\mathcal{O}}$ of tasks in $\mathcal{G}_n$. The binary variable $x_{v_{n,j}}^{k}$ is set to 1 if task $v_{n,j}$ is selected for the k-th position of the schedule, and 0 otherwise. Constraint (C5) guarantees that each task is scheduled only once, ensuring a total of $\mid \mathcal{V}_n \mid$ schedules and that only one task is selected for each schedule. Constraint (C6) defines the offloading matrix $\mathbf{X}_{\mathcal{V}}^{\mathcal{S}}$ for the task set $\mathcal{V}_n$. The notation $x_{v_{n,j}}^{s_b} = 1$ indicates that the task $v_{n,j}$ is offloaded to device s_b, and it equals 0 otherwise. Constraint (C7) states that task $v_{n,j}$ can be offloaded to only one device. Constraint (C8) defines the transmit power allocation matrix and gives the range of the transmit power of the user equipment. Constraint (C9) establishes the computational resource allocation matrix $\mathbf{F}_{\mathcal{V}}^{\mathcal{S}}$ for the task set $\mathcal{V}_n$, and specifies the allowable range of the clock frequency $f_{v_{n,j}}^{s_b}$ assigned to task $v_{n,j}$, which must be between f_{min} and $f_{cap}^{s_b}$. Constraint (C10) ensures that the total clock frequency allocated to all tasks on device s_b does not exceed its maximum capacity $F_{cap}^{s_b}$. Together, these constraints form a comprehensive framework that guarantees the feasibility of the optimization problem.

As outlined in [43], a decision problem **H** classified as NP-hard if a polynomial-time reduction exists from an NP-complete problem **C** to **H**. Thus, the NP-hardness proof process which will be employed in this paper consists of the following steps: 1) Identify an NP-complete problem **C**. 2) Define the problem **H**. 3) Develop a transformation $\mathtt{F}$ from **C** to **H**. 4) Proof that $\mathtt{F}$ is a polynomial-time transformation. Consequently, we establish the following theorem.

Theorem 1. *Problem (14) is NP-hard.*

Proof. The general scheduling problem is an NP-complete problem [44], which can be formally defined as follows.

Problem **C**: Given a set S of n jobs, a partial order $\prec$ on S, a weight function $\mathtt{W}$, a number of processors k and a time limit l. Does there exist a total function $\mathtt{F}$ from S to $\{0, 1, \cdots, l-1\}$ such that

(1) if $v \prec v'$, then $\mathtt{F}(v) + \mathtt{W}(v) \leq \mathtt{F}(v')$,

(2) for each v in S, $\mathtt{F}(v) + \mathtt{W}(v) \leq l$,

(3) for each $i, 0 \leq i < l$, there are at most k values of v for which $\mathtt{F}(v) \leq i < \mathtt{F}(v) + \mathtt{W}(v)$.

To simplify the argument, we consider a special case of (14) where we ignore the energy consumption objective and focus on the application duration objective. Additionally, we restrict the optimization variables to only consider task offloading decisions. This case (problem **H**) represents the scheduling of a single DAG application with strict precedence constraints and limited parallelism, which can be described as:

$$\min_{X_{\mathcal{V}}^{\mathcal{S}}} T_{\mathcal{G}_n}$$

We establish the NP-hardness by reducing problem **C** to problem **H**, demonstrating that problem **H** is at least as hard as problem **C**.

Mapping the Inputs: Problem **C** takes the same inputs as Problem **H** because a partial order on tasks is equivalent to a DAG. In a partial order, if task v must be executed before task v' ($v \prec v'$), this can be (v, v') in a DAG. The number of available processors in Problem **C** maps directly to the resource constraints in problem **H**.

Constructing the Reduction: For problem **C** with time limit l, we solve problem **H** to to determine the optimal duration $T_{\mathcal{G}_n}$.

- if $T_{\mathcal{G}_n} \leq l$, then the answer to problem **C** is yes.
- if $T_{\mathcal{G}_n} > l$, then the answer to problem **C** is no.

Since problem **C** is NP-hard, and we have shown that problem **C** reduces to problem **H**, it follows that problem **H** is also NP-hard. Finally, as problem (14) generalizes problem **H** by introducing additional objectives and variables, problem (14) is at least as hard as problem **H**. Hence, problem (14) is NP-hard. □

V. Algorithm

In this section, we present the design of our proposed algorithm. First, we describe the state space, action space, and reward function in the context of a deep reinforcement learning model. Next, we detail the design of the agent, focusing on state representation and action generation. Finally, we outline the training process.

A. Deep Reinforcement Learning Model

The reinforcement learning framework constitutes a general approach for decision-making problems formalized as Markov decision processes (MDPs). By incorporating deep learning techniques, DRL facilitates the efficient extraction of relevant features from high-dimensional observations, enabling agents to learn optimal policies. Within this framework, the agent interacts with the environment through a sequential process. The entire process is divided into several periods, called time step $\boldsymbol{t} \in \mathbb{T}$. At time step $\boldsymbol{t}$, the DRL agent observes the current state $\boldsymbol{s_t}$, takes an action $\boldsymbol{a_t}$, obtains an immediate reward $\boldsymbol{r_t}$, and transitions to the next state $\boldsymbol{s_{t+1}}$ according to the environment dynamics $\boldsymbol{p(s_{t+1} \mid s_t, a_t)}$. The policy $\boldsymbol{\pi_\theta}$, which maps states to actions, is represented by a neural network with parameters $\boldsymbol{\theta}$. The DRL agent aims to maximize its expected cumulative reward $R_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$, where $\gamma \in (0, 1]$ is a discount factor.

Given that the joint optimization problem given by (14) constitutes a sequential decision process, we model it as an MDP and design the corresponding state space, action space, and reward function to facilitate a DRL-based solution.

State Space. The state $\boldsymbol{s_t}$ represents the agent's observation of the edge networking environment at time step $\boldsymbol{t}$, providing the necessary information to guide the agent's decision-making for the next time step. To effectively address the joint problems outlined in Section IV-C, the state $\boldsymbol{s_t}$ should encapsulate a comprehensive set of factors, including the status of the communication network, computing node resource information,

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on November 05,2025 at 08:36:05 UTC from IEEE Xplore. Restrictions apply.
© 2025 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Wireless Communications. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TWC.2025.3619100

application topology information, and so on. Consequently, $\mathbb{S}$ is denoted as:

$$\mathbb{S} = \{\mathcal{N}, \mathcal{M}, \mathcal{G}, \mathcal{L}\}, \tag{15}$$

where $\mathcal{N}$ denotes the set of user devices, including information about user equipment status and resource availability. $\mathcal{M}$ denotes the set of edge servers, which includes the number of CPU resource capacity $F_{cap}^{s_m}$, CPU utilization rates, and other relevant details. $\mathcal{G}$ captures the DAG application information, including the dependencies between tasks, data volume, and other task attributes such as the required CPU resources, the percentage of task completion, the current task ID, and the associated application ID. Finally, $\mathcal{L}$ encompasses the environmental information, including the channel gain of the link from the user n to the base station m and other relevant environmental conditions that affect the communication quality.

Action Space. In our DAG scheduling problem within edge networks, a key challenge in this context is the vast action space created by managing task priorities and allocating resources. The action space consists of multiple decision variables, including application selection, task selection, device selection, transmission power allocation, and computational resource allocation. A direct approach would be to generate all action dimensions simultaneously. However, this results in a combinatorial explosion in the search space, making optimization intractable.

To tackle this challenge, we design a structured and hierarchical action space to mitigate the complexity of individual decisions. By organizing and layering the action space, we significantly reduce its complexity. This hierarchical approach allows agents to make decisions sequentially, breaking down the decision-making process into the five manageable stages described above. At each stage, invalid operations are filtered out during the generation of actions, thereby allowing agents to converge faster. The action at time step $\boldsymbol{t}$ is defined as:

$$\boldsymbol{a_t} = \{\boldsymbol{a}_{\mathcal{G}_n}, \boldsymbol{a}_{v_{n,j}}, \boldsymbol{a}_{s_b}, \boldsymbol{a}_p, \boldsymbol{a}_f\}. \tag{16}$$

The action space is structured into five dimensions. 1) DAG application selection $\boldsymbol{a}_{\mathcal{G}_n}$: Integer values $\{1, 2, \ldots, N\}$, where each integer corresponds to a unique DAG application. 2) Task selection $\boldsymbol{a}_{v_{n,j}}$: Integer values $\{1, 2, \ldots, \max\{|G_n|\}\}$, where each integer corresponds to a unique task, and the order must respect task dependencies defined in the DAG. 3) Offloading device selection $\boldsymbol{a}_{s_b}$: Integer values $\{0, 1, 2, \ldots, M\}$, where 0 represents the local device, and $\{1, 2, \ldots, M\}$ correspond to edge servers. 4) Transmit power allocation $\boldsymbol{a}_p$: Discrete values from a predefined set $\mathcal{P} = \{P_1, P_2, \ldots, P_{L_p}\}$, where $P_1 = P_{\min}$ and $P_K = P_{\max}$. L_p is the discrete level of user transmission power. 5) Computational resource allocation $\boldsymbol{a}_f$: Discrete values from a predefined set $\mathcal{F} = \{f_1, f_2, \ldots, f_{L_f}\}$, where $f_1 = f_{\min}$ and $f_L = \max\{F_{cap}^{s_b}\}$. Similar to transmit power, L_f is the discrete level of CPU frequency resources.

Reward Function. To ensure that the agent always selects feasible actions during training, we have introduced an action masking mechanism. At each time step, before the agent selects an action, it first evaluates all possible actions and checks whether they satisfy the constraint conditions (C1 $\sim$ C10). If an action violates any of these constraints, its probability is set to zero, meaning the agent cannot choose that action. As a result, the agent will only select from actions that satisfy the constraints, thus avoiding the exploration of infeasible solutions.

As described in (14), the optimization objective in this article is to minimize the average weighted cost Ω of DAG applications. To keep the agent consistent with this goal, we designed the reward function $\boldsymbol{r_t}$ to guide its actions directly. Specifically, the reward $\boldsymbol{t}$ for each time step is defined as:

$$\boldsymbol{r_t} = -\Omega, \tag{17}$$

where Ω represents the weighted cost of users.

B. Design of Transformer-based Agent

Graph neural networks (GNNs) [45] have been widely used for processing graph-structured data. However, when applied to DAGs, traditional message-passing mechanisms limit GNNs' ability to fully leverage the characteristics of DAGs, such as node depth and topological ordering. We proposed a new agent based on the transformer architecture to address these limitations. The transformer model is a powerful tool for processing sequential data and has been widely used in natural language processing tasks. The key idea behind the transformer is to use a self-attention mechanism to model the relationships between different elements in the input sequence. The transformer architecture consists of several key components, including the self-attention mechanism, feed-forward network (FFN), and layer normalization, which is defined as:

$$\begin{aligned} \text{Attn}(\mathbf{X}) &:= \text{softmax}(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_{out}}})\mathbf{V} \in \mathbb{R}^{n \times d_{out}}, \\ \mathbf{X}' &= \mathbf{X} + \text{Attn}(\mathbf{X}), \\ \mathbf{X}'' &= \text{FFN}(\mathbf{X}') := \text{ReLU}(\mathbf{X}'\mathbf{W}_1)\mathbf{W}_2. \end{aligned} \tag{18}$$

Based on the transformer architecture, we introduce a new attention mechanism [46]. The new attention mechanism is designed to selectively focus on the relevant parts of the graph, such as the direct predecessors and successors of each node, rather than all nodes. The new self-attention mechanism is defined as:

$$\text{Attn}_{\text{DAG}}(x_i) = \sum_{j \in N_k(i)} \frac{\kappa(x_i + P_E^i, x_j + P_E^j)}{\sum_{w \in N_k(i)} \kappa(x_i + P_E^i, x_w + P_E^w)} f(x_j), \tag{19}$$

where x_i is the feature vector of node i, $N_k(i)$ is the set of k-th order neighbors of node i, κ is a learnable similarity function, P_E^i is the positional encoding of node i, and f is a non-linear transformation function. The positional encoding P_E^i reflects the depth of node i in the DAG:

$$P_E^v = \begin{cases} \sin\left(\frac{\text{depth}(v_i)}{10000^{2i/d}}\right) & \text{if } i \text{ is even}, \\ \cos\left(\frac{\text{depth}(v_i)}{10000^{2i/d}}\right) & \text{if } i \text{ is odd}, \end{cases} \tag{20}$$

where d is the dimensionality of the node features, and i is the index of the feature dimension. depth$(\cdot)$ is used to determine the depth of node v_i in the DAG relative to the virtual entry node v_0, where each node has a unique depth.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on November 05,2025 at 08:36:05 UTC from IEEE Xplore. Restrictions apply.
© 2025 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Wireless Communications. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TWC.2025.3619100

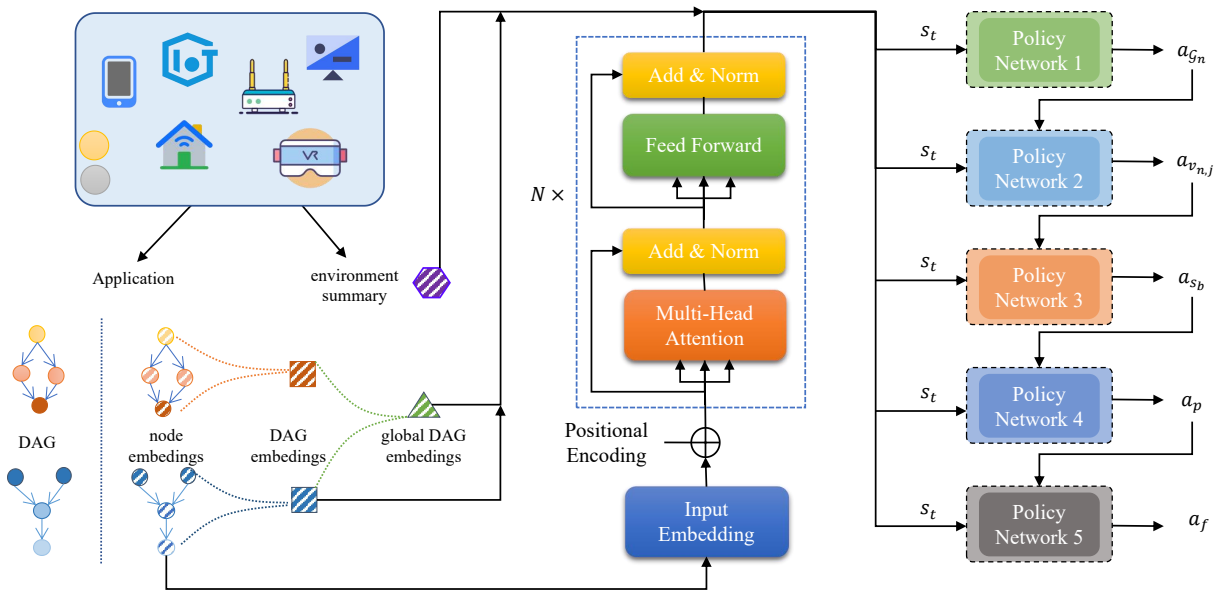


Fig. 2. Design of the transformer-based agent.

In our design, the agent captures the composition state of different aspects of the DAG and the entire system, as shown in Fig. 2. Here is a detailed breakdown:

- Node embeddings: Each node $v_{n,j}$ in the DAG is represented by a feature vector, which encodes both the node's characteristics and contextual information from its neighbors. These embeddings are computed using a Transformer-based neural network with multi-head attention and positional encoding.
- DAG embeddings: The DAG embedding is obtained by aggregating the embeddings of all nodes within a DAG using a max-pooling operation, providing a compact representation of the entire DAG.
- Global DAG embeddings: The embeddings of all active DAGs are aggregated using a max-pooling operation across DAGs. This provides an overview of the global execution status of the applications in the system.
- Environment summary: This component encapsulates additional system information, including resource availability, communication network conditions, and other external factors that influence scheduling decisions.

Agents generate actions based on a hierarchical approach, where each policy network is conditioned on the actions output by the previous policy network. Specifically, the first-level policy network determines which dag application to process. Subsequent policy layers meticulously refine the decision-making process, including which tasks to prioritize, which equipment to select, and how many resources to allocate to each task. Through this thorough layer-by-layer refinement method, we significantly reduce the complexity of agent decisions in high-dimensional space and achieve efficient action generation. The action generation formula is:

$$\boldsymbol{a_t^i} = \boldsymbol{f_t^i}(\boldsymbol{a_t^{i-1}}, \boldsymbol{s_t}), 1 \leq i \leq |\mathbb{A}|, \ \boldsymbol{a_t^0} = \varnothing, \tag{21}$$

where $\boldsymbol{a_t^{i-1}}$ is the sampled action of the policy network $\boldsymbol{f_t^i}(\cdot)$ at time step $\boldsymbol{t}$.

C. Training Process

This study used a PPO to train our agent. PPO is a variant of the actor-critic approach that effectively balances exploration and exploitation. It decomposes the learning problem into two components: actor, which selects the action based on the current policy, and critic, which evaluates the value of the current state. Specifically, the actor-network parameterized by ϑ determines the optimal action to take in a given state, while the critic-network parameterized by φ estimates the value of the state to guide policy updates.

The objective is to maximize the expected return $J(\vartheta)$ under the policy φ_ϑ, defined as the expected total discounted return of all trajectories $\boldsymbol{\tau}$:

$$J(\vartheta) = \mathbb{E}_{\tau\sim\pi_\vartheta}[R(\tau)] = \mathbb{E}_{\tau\sim\pi_\vartheta}\left[\sum_{t=0}^{T}\gamma^t \boldsymbol{r_t}\right], \tag{22}$$

where ϑ represents the parameters of the policy, $\boldsymbol{\tau}$ is a complete trajectory, γ is the discount factor, and $\boldsymbol{r_t}$ is the immediate reward at time step $\boldsymbol{t}$. The optimal policy parameters ϑ^* are those that maximize $J(\vartheta)$:

$$\vartheta^* = \arg\max_{\vartheta} J(\vartheta). \tag{23}$$

To reach our goal, we move along the direction of the gradient $\nabla_\vartheta J(\vartheta)$ to update the policy parameters ϑ. The aim is to find a new set of parameters ϑ' such that $J(\vartheta') > J(\vartheta)$. The improvement in the objective function can be quantified as:

$$J(\vartheta') - J(\vartheta) = \mathbb{E}_{\tau\sim\pi'}\left[\sum_{t=0}^{T}\gamma^t A^\pi(\boldsymbol{s_t}, \boldsymbol{a_t})\right], \tag{24}$$

$$A^\pi(\boldsymbol{s_t}, \boldsymbol{a_t}) = Q^\pi(\boldsymbol{s_t}, \boldsymbol{a_t}) - V^\pi(\boldsymbol{s_t}). \tag{25}$$

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on November 05,2025 at 08:36:05 UTC from IEEE Xplore. Restrictions apply.
© 2025 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Wireless Communications. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TWC.2025.3619100

Here, $A^{\pi}(\boldsymbol{s_t}, \boldsymbol{a_t})$ is the advantage function, which measures how better it is to take an action $\boldsymbol{a_t}$ in the state $\boldsymbol{s_t}$ than to choose an action randomly. It indicates whether an action should be encouraged or discouraged. This information is critical for updating the policy. To reduce variance and improve data efficiency, we use generalized advantage estimation (GAE) for estimating the advantage function:

$$\begin{aligned} A^{\pi}_{\text{GAE}}(\boldsymbol{s_t}, \boldsymbol{a_t}) &= \sum_{\ell=0}^{\infty} (\gamma\lambda)^{\ell} \delta_{t+\ell}, \\ \delta_t &= \boldsymbol{r_t} + \gamma V^{\pi}(\boldsymbol{s_{t+1}}) - V^{\pi}(\boldsymbol{s_t}), \end{aligned} \tag{26}$$

where γ is the discount factor and λ controls the trade-off between bias and variance. GAE allows the agent to have a more accurate estimate of the advantage by smoothing out the temporal difference errors over multiple steps, which leads to lower variance in the gradients.

When the new and old policies are relatively close, their state distribution is similar. In this case, we can approximate the improvement in the objective function using trajectories generated by the old policy π and adjust for the difference using importance sampling. The improvement can then be approximated as:

$$\begin{aligned} J(\vartheta') - J(\vartheta) &\approx \mathbb{E}_{\tau\sim\pi}\left[\sum_{t=0} \frac{\pi'(\boldsymbol{a_t} \mid \boldsymbol{s_t})}{\pi(\boldsymbol{a_t} \mid \boldsymbol{s_t})} A^{\pi}_{\text{GAE}}(\boldsymbol{s_t}, \boldsymbol{a_t})\right] \\ &= \mathbb{E}_t\left[p_t(\vartheta) A_t\right]. \end{aligned} \tag{27}$$

To ensure that the policy updates do not deviate too much from the previous policy, thus maintaining training stability, we employ a clipping mechanism on the importance sampling ratio. The policy update is given by:

$$\begin{aligned} J^{\text{clip}}(\vartheta) = \mathbb{E}_t\Big[\min\big(&p_t(\vartheta) A^{\text{GAE}}_t, \\ &\text{clip}(p_t(\vartheta), 1-\varepsilon, 1+\varepsilon) A^{\text{GAE}}_t\big)\Big], \end{aligned} \tag{28}$$

where ε is a hyperparameter defining the clipping neighborhood $|p_t(\vartheta) - 1| \leq \varepsilon$. While considering the training of the critic network, the overall objective function for the agent takes the following form:

$$J^{\text{C+H+V}}(\vartheta) = J^{\text{clip}}(\vartheta) + \omega_h H - \omega_v J^{\text{val}}(\vartheta), \tag{29}$$

where ω_h is a weight factor for the entropy bonus H, which encourages an increase in the policy's action entropy, thereby ensuring adequate exploration throughout the training process. Similarly, ω_v is also a weighting factor. The term $J^{\text{val}}(\vartheta) = \mathbb{E}_t\left[\left(V_t^{target} - V(\boldsymbol{s_t})\right)^2\right]$ is the square error value function used to train the critic model, measuring the difference between the target value V_t^{target} and the estimated value $V(\boldsymbol{s_t})$.

The detail of the training process in our proposed approach is show in Algorithm 1. Lines 1-4 initialize the hyperparameters, as well as the actor and critic networks. Lines 5-6 observe the state and obtain the feature embeddings. Lines 7-11 describe how the transformer-based agent collects trajectories by sequentially generating actions. Lines 13-22 outline the training loop, including the update mechanisms for both the actor and critic networks.

Algorithm 1: Transformer-based Agent Training Algorithm for Integrated Application Ordering, Task Prioritization, Offloading, Transmit power Allocation, and Computational Resource Allocation

Data: Set of user devices $\mathcal{N}$; set of edge servers $\mathcal{M}$; DAG application information $\mathcal{G}$; environmental information $\mathcal{L}$.

Result: The policy network π_{ϑ}.

1 Set appropriate hyper-parameters such as actor learning rate η_a, critic learning rate η_c, discount factor γ, and clipping parameter ε ;
2 Initialize the actor network $\pi_{\vartheta}(\boldsymbol{a_t} \mid \boldsymbol{s_t})$; initialize the critic network $\pi_{\varphi}(\boldsymbol{a_t} \mid \boldsymbol{s_t})$;
3 **for** $i = 1, 2, \cdots, \mid B \mid$ **do**
4 Initialize old actor network with $\pi'_{\vartheta} = \pi_{\vartheta}$;
5 Get state $\boldsymbol{s_t}$ from observation and calculate the node embeddings and positional encoding P_E^i;
6 Get the graph embeddings, global graph embeddings, and environment summary ;
7 The transformer-based agent uses policy π'_{ϑ} to sequentially generate actions for application selection, task prioritization, offloading decision, transmit power allocation and computational resource allocation;
8 Interacts with the environment and collect a set of trajectories $\boldsymbol{\tau} = (\boldsymbol{s_t}, \boldsymbol{a_t}, \boldsymbol{r_t}, \boldsymbol{s_{t+1}})$;
9 Calculate advantage A^{π} and target value V^{π}_{tar} using actor network π'_{ϑ}, critic network π_{φ}, and trajectory data;
10 **end**
11 Let batch B consist of the collected trajectories, advantages, and target values;
12 **for** $i = 1, 2, \cdots, T$ **do**
13 **for** minibatch m in batch B **do**
14 Calculate the probability ratio $p(\vartheta)$ between the old and new policies;
15 Calculate the clipped objective $J^{\text{clip}}(\vartheta)$ by (28);
16 Calculate the policy entropy H using the actor network π_{ϑ};
17 Calculate the policy loss: $L_p(\vartheta) = J^{\text{clip}}(\vartheta) + \omega_h H$;
18 Update the actor network parameters ϑ: $\vartheta \leftarrow \vartheta + \eta_a \nabla_{\vartheta} L_p(\vartheta)$;
19 Calculate the predicted value V using the critic network π_{φ};
20 Calculate the value loss: $L_v(\varphi) = \text{MSE}(V, V^{\pi}_{tar})$;
21 Update the critic network parameters φ: $\varphi \leftarrow \varphi - \eta_c \nabla_{\varphi} L_v(\varphi)$;
22 **end**
23 **end**

VI. Evaluation

This section provides a detailed evaluation of the proposed algorithm based on simulation results. We first describe the experimental setup and simulation parameters, followed by a

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on November 05,2025 at 08:36:05 UTC from IEEE Xplore. Restrictions apply.
© 2025 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Wireless Communications. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TWC.2025.3619100

TABLE II
SUMMARY OF THE SIMULATION PARAMETERS

Parameters	Definition	Values
W	Bandwidth	80 MHz
λ	application arrival rate	0.04
N_{attn}	number of attention heads	4
N_{en}	number of encoder layers	4
f_a	activation function of actor	$Tanh$
N_{iter}	number of iterations	1200
η_a	learning of actor network	5×10^{-4}
η_c	learning of critic network	3×10^{-5}
ε	clip range	0.2

performance comparison with multiple baseline algorithms.

A. Experimental Setup

We utilized Gymnasium [47] to implement an event-driven, simulated edge computing environment. The simulation environment was configured with 500 users and 10 edge servers during the training phase. User arrivals followed a Poisson process with an arrival rate of $\lambda = 0.04$, associated with a single application. The DAG application structures were derived from the Alibaba's cluster trace [48]. The CPU cycle frequencies of user devices and edge servers were set to 0.5 GHz and 1.6 GHz, respectively. The computational requirements of tasks ranged from 20 to 300 million cycles, while the data size for each edge varied between 10 and 200 Mb. The number of tasks within an application was determined by its structure. Table II summarizes the additional simulation parameters used in our study.

We implemented and trained our event-driven PPO algorithms using PyTorch, a deep learning framework. The PyTorch Geometric library [49] was leveraged to accelerate data loading, training, and reasoning for DAGs, significantly improving computational efficiency. The experiments were conducted on a server equipped with a single Intel i9-10900 CPU and a single NVIDIA RTX 3060 GPU, running 20 parallel training environments. The software environment consisted of Python 3.10, PyTorch 2.2.1, PyTorch Geometric, and the Debian 12 operating system.

B. Performance Evaluation

Fig. 3 demonstrates the influence of different learning rates on the convergence of the model. By adjusting the learning rate, $\eta \in \{5e^{-2}, 5e^{-3}, 5e^{-4}, 5e^{-5}\}$, maintaining other conditions constant, we monitor the training progress of the model. We find that a moderate learning rate ($\eta = 5e^{-4}$) rapidly approaches the optimal performance within an ideal time frame. Specifically, Fig. 3(b) shows that this setting has the shortest average application duration, indicating optimal performance. Fig. 3(c) reveals that energy consumption rises due to increased resource utilization locally. Conversely, an excessively high learning rate ($\eta = 5e^{-2}$) leads to training instability and difficulty in convergence. Fig. 3(b) indicates that this setting has the longest average application duration. Fig. 3(c) supports this claim by showing inefficient use of local device resources, resulting in lower average energy consumption. On the other hand, a low learning rate ($\eta = 5e^{-5}$) gets stuck in a local optimum, as evidenced by the slow increase in total weighted reward and the inability to achieve the optimal average application duration.

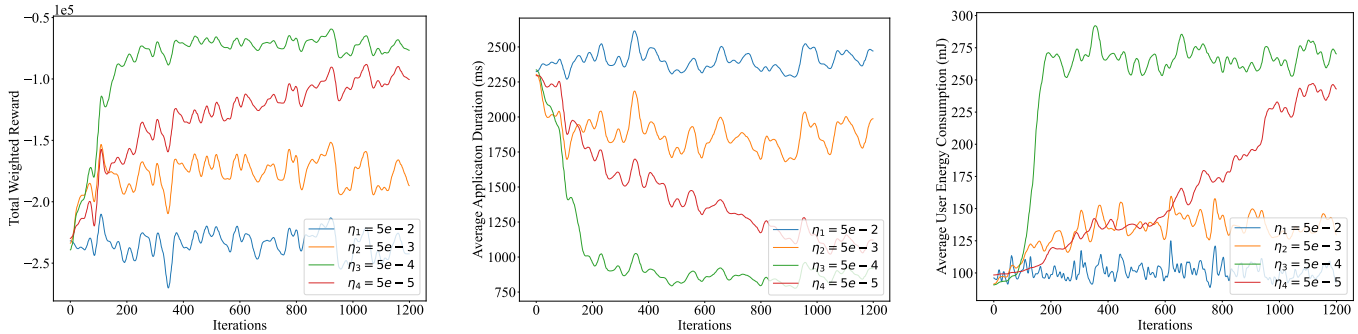
Fig. 4 compares the performance of our proposed PPO-based algorithm against two baseline reinforcement learning methods: Vanilla Policy Gradient (VPG) and Advantage Actor-Critic (A2C). VPG relies solely on policy gradient updates without value estimation, while A2C integrates a critic network to provide bootstrapped advantage signals for more stable learning. Under continuous DAG task arrivals, VPG struggles to converge due to high variance in gradient estimates. In contrast, both A2C and PPO benefit from the critic's guidance, which enables more reliable policy updates and faster convergence. Notably, PPO further enhances training stability through its clipped objective function, which constrains policy updates to avoid destructive changes and encourages exploration toward higher-performing policies. As shown in Fig. 3(b), PPO achieves significantly shorter average application duration compared to both A2C and VPG. Furthermore, as illustrated in Fig. 3(c), PPO not only reduces energy consumption relative to A2C but also maintains superior latency performance — demonstrating its ability to better balance efficiency and responsiveness in dynamic environments.

Fig. 5 shows the convergence behavior and performance of our proposed algorithm under different experimental settings. Fig. 5(a) illustrates the convergence across multiple random seeds. The dashed lines represent individual training trajectories, while the solid line and shaded region correspond to the mean performance and standard deviation, respectively. The results show that our algorithm consistently improves over iterations, with a stable convergence trend. The relatively low variance across seeds suggests robustness against initialization randomness, ensuring reliable performance in different training scenarios. Fig. 5(b) presents the performance of our proposed algorithm during the training phase when different neural networks are employed for feature extraction. Specifically, our method incorporates a transformer-based feature extractor, whereas a GNN and a fully connected neural network (FCN) serve as comparison models. As observed, the transformer-based feature extractor demonstrates superior convergence speed and stability compared to both the GNN and FCN.

Fig. 6 evaluates the robustness of our algorithm under varying arrival rates $\lambda \in \{0.02, 0.04, 0.06, 0.08, 0.10\}$, comparing the inference performance of agents based on FCN, GNN, and Transformer architectures. As shown in Fig. 6(a), the Transformer-based agent consistently achieves the lowest application duration across all tested λ values, demonstrating superior generalization and robustness to system load variations. In contrast, both FCN and GNN exhibit significant performance degradation as λ increases, with FCN suffering the most severe deterioration. Similarly, Fig. 6(b) shows that while energy consumption generally rises with λ, the Transformer-based agent maintains the most efficient energy profile throughout the range, outperforming its counterparts especially at higher loads. The results of this sensitivity analysis demonstrate that while all models degrade under increasing load, the Transformer-based agent maintains superior perfor-

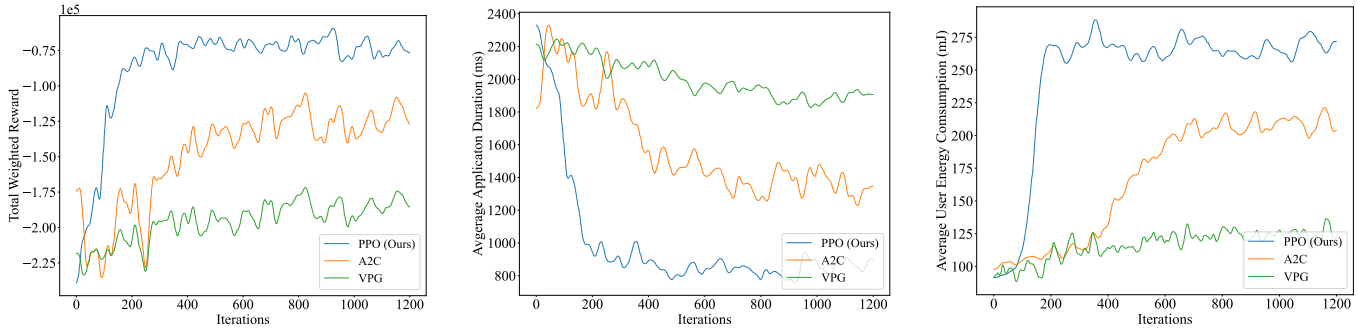
Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on November 05,2025 at 08:36:05 UTC from IEEE Xplore. Restrictions apply.
© 2025 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Wireless Communications. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TWC.2025.3619100



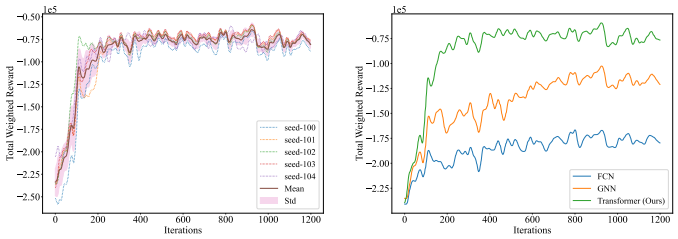
(a) Weighted Reward (b) Application Duration (c) Energy Consumption

Fig. 3. The values of the total weighted reward, average application duration, and average user energy consumption during the training process of the proposed algorithm at various learning rates.



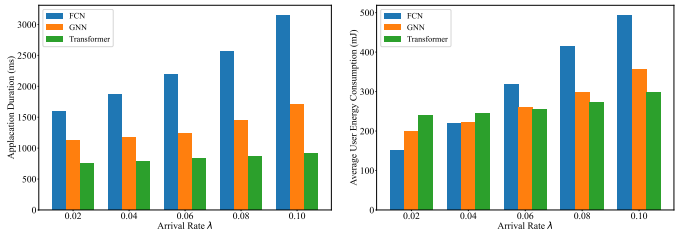
(a) Weighted Reward (b) Application Duration (c) Energy Consumption

Fig. 4. The values of the total weighted reward, average application duration, and average user energy consumption during the training process of the proposed algorithm with other reinforcement learning algorithms.



(a) Different random seeds (b) Different neural networks

Fig. 5. Convergence Trends Under Different Seeds and Feature Extraction Strategies.



(a) Application Duration (b) Energy Consumption

Fig. 6. Inference performance of agents based on different neural networks under varying arrival rates.

mance and efficiency across the entire range, highlighting its robustness and practical advantage in unpredictable system conditions.

Overall, the comparison between Fig.6(a) and Fig.6(b) demonstrates that the Transformer-based model generalizes more effectively under varying arrival rates, maintaining superior performance and stability compared to the other architectures.

In Fig. 7 , by adjusting the balance factor α within the range of $[0.1, 0.9]$, a trade-off is observed between application duration and energy consumption. This demonstrates the flexibility of the proposed scheme, which can effectively balance performance based on the varying demands of time-sensitive and energy-saving applications.

Fig. 8 compares the performance of different task prioritization algorithms used in the training phase, with the resource allocation algorithm consistently set to the DRL method. Specifically, the critical path method (CPM) denotes that tasks on the critical path within the DAG are executed first. The first come first serve (FCFS) represents an approach where tasks are processed according to their arrival times. Lastly, the random policy signifies the random selection of tasks for processing. As demonstrated in Fig.8, our proposed scheme demonstrates superior performance compared to CPM and FCFS. In contrast, the Random scheme exhibits the lowest performance. Notably, the Random scheme exhibits particularly poor performance, as its arbitrary task selection approach significantly prolongs task completion and leads to

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on November 05,2025 at 08:36:05 UTC from IEEE Xplore. Restrictions apply.
© 2025 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Wireless Communications. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TWC.2025.3619100

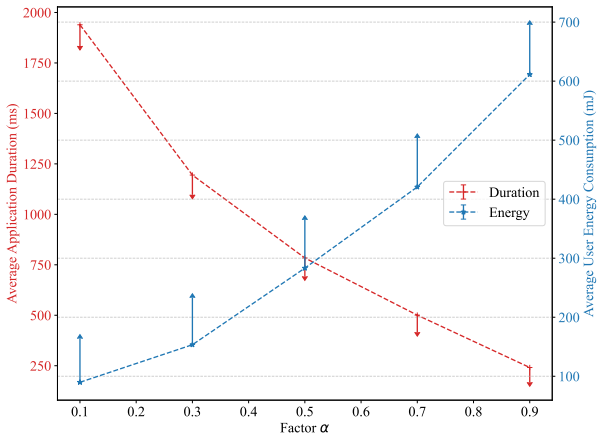


Fig. 7. Balance factor α

prolonged application delays.

Fig. 9 illustrates the performance comparison of various task placement algorithms during the training process, while keeping our task prioritization algorithm fixed. Specifically, the Fair scheme allocates resources evenly among all tasks in the system. The Greedy scheme allocates as many resources as possible to the selected tasks, and the weighted scheme allocates resources based on demands of currently executable tasks. As indicated in Fig. 9, we can conclude that our proposed scheme demonstrates superior performance relative to the Greedy scheme. One possible reason is that the Greedy scheme does not consider the data transmission time between tasks. our proposed scheme demonstrates superior performance relative to the Greedy scheme. One plausible explanation for the Greedy scheme's relatively poorer performance is that it excessively allocates resources to individual tasks without effectively balancing resource distribution among concurrent tasks, resulting in resource contention and longer overall execution times. The Fair and Weighted schemes perform the worst since the system has many parallel tasks, each of which can only be allocated to a minimum amount of resources (i.e., f_{min}). As a result, the curves of the two schemes are very similar.

Fig. 10 illustrates the comparative performance of the presented scheme against various heuristic algorithms in terms of application duration and energy consumption. Our method reduces the application duration to 738.22 ms, achieving a 14.5% improvement over the CPM+Greedy method, which stands at 863.12 ms. Furthermore, It also cuts energy consumption to 297.94 mJ, marking a 10.4% reduction from CPM+Greedy's 332.48 mJ. Additionally, the error bars reveal that our method exhibits smaller standard deviations compared to other algorithms, indicating better stability.

The previous experiments have demonstrated the advantages of the proposed algorithm in specific aspects. In the following, we further evaluate its performance under different environmental settings by comparing it with several baseline approaches:

- Ours: The algorithm proposed in this work;
- FCFS: Executes tasks in the order of their arrival times;
- SJF: Prioritizes application with the shortest execution time;
- CPOP: The Critical-Path-on-a-Processor algorithm targets scheduling of all critical tasks onto a single processor [50];
- Tetris: The Tetris scheduling algorithm optimizes task placement and execution by matching resource demands and constraints [51].

Fig. 11 shows a performance comparison of five algorithms across different numbers of DAG applications. As depicted in Fig. 11(a), the average application duration tends to increase with a growing number of applications. However, our proposed method demonstrates a significantly smaller increase in application duration compared to other approaches, indicating better scalability. Fig. 11(b) depicts the trends in energy consumption with increasing application volumes. While energy consumption increase for all strategies, our method consistently exhibits the lowest rate of increase, highlighting its superior energy efficiency. In comparison to the CPOP method, our approach achieves a reduction in duration ranging from 4.9% to 9.2%, and a decrease in energy consumption of 6.1% to 12.5%. These results collectively demonstrate the effectiveness and scalability of our scheduling strategy, particularly in the context of large-scale application environments.

Fig. 12 compares the performance of the five algorithms in edge server environments with different computing resource configurations. As shown in Fig. 12(a) and Fig. 12(b), when the resources of the edge server are more abundant, the average application latency is significantly reduced, and the average energy consumption of users is also reduced. The increase in edge server resources reduces the computational load local devices need to handle, thus reducing overall energy consumption. In addition, no matter how the resources of the edge server change, the proposed algorithm can maintain its optimal performance.

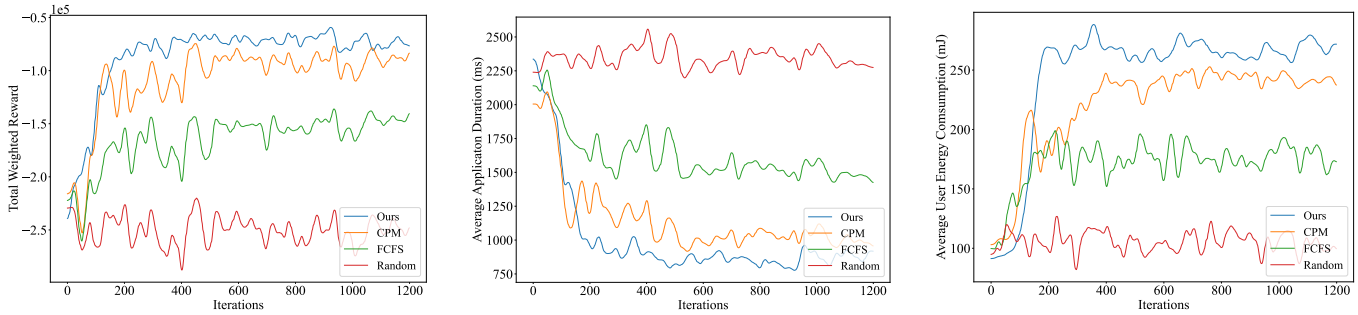
VII. Conclusion

In this paper, we investigated the problem of intelligent application scheduling in multi-user edge networks. The key challenges stemmed from the dynamic nature of the environment, constrained computational resources, and the stochastic arrival of applications. To address these challenges, we proposed a novel DRL-based scheduling framework that adaptively assigned interdependent tasks to appropriate computing devices and dynamically allocates resources. A key feature of our approach was the transformer-based feature extraction network, which enhanced the agent's understanding of task dependencies and improved scheduling decisions. Additionally, our algorithm effectively balanced the trade-off between energy consumption and execution latency, ensuring efficient resource utilization in edge computing environments. Extensive simulation results demonstrated that our framework achieves stable convergence. Furthermore, across various test scenarios, our approach reduces application duration by up to 9.2% and energy consumption savings by up to 12.5% compared to the CPOP method.

For future work, we plan to extend our framework in two key directions. On the one hand, we will investigate decen-

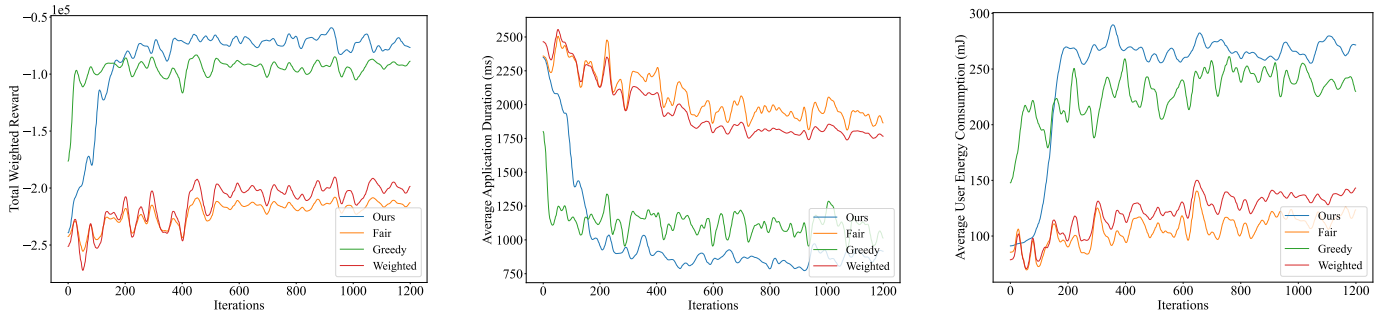
Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on November 05,2025 at 08:36:05 UTC from IEEE Xplore. Restrictions apply.
© 2025 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Wireless Communications. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TWC.2025.3619100



(a) Weighted Reward (b) Application Duration (c) Energy Consumption

Fig. 8. Convergence performance of different task priority algorithms.



(a) Weighted Reward (b) Application Duration (c) Energy Consumption

Fig. 9. Convergence performance of different task placement strategies

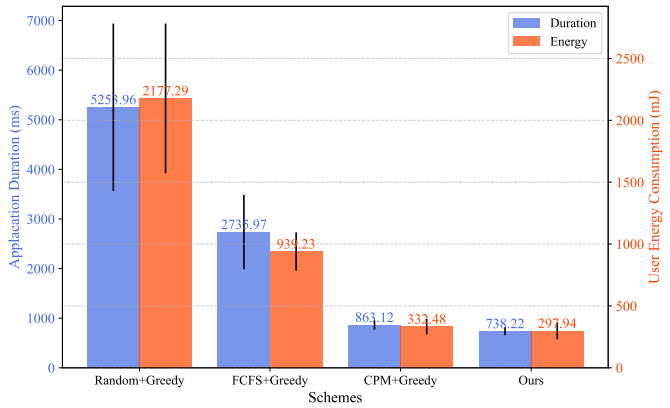
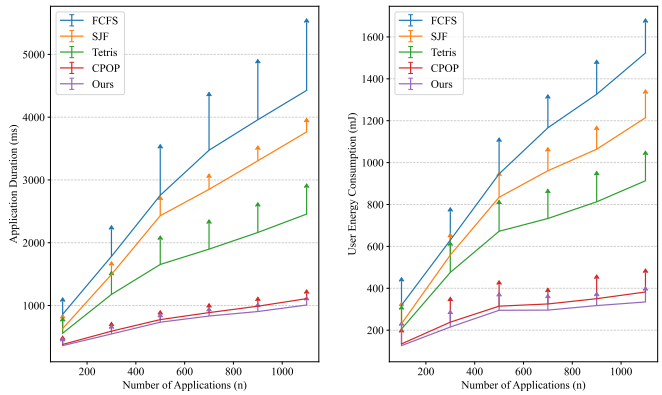


Fig. 10. The performance comparison for different heuristic algorithms and the proposed algorithm.



(a) Application Duration (b) Energy Consumption

Fig. 11. The performance comparison for different number of applications.

tralized scheduling methods that operate effectively without requiring full global information, enabling practical deployment in highly distributed edge networks. On the other hand, we will study scheduling performance in scenarios involving high-speed user mobility, focusing on environments with rapid topology changes and intermittent connections.

References

[1] S. S. Sahoo, S. Mohanty, K. S. Sahoo, M. Daneshmand, and A. H. Gandomi, "A three-factor-based authentication scheme of 5g wireless sensor networks for iot system," *IEEE Internet of Things Journal*, vol. 10, no. 17, pp. 15 087–15 099, 2023.

[2] P. Gope, B. Sikdar, and N. Saxena, "Guest editorial: Security and privacy in 5g-enabled industrial iot current progress and future challenges," *IEEE Transactions on Industrial Informatics*, vol. 19, no. 1, pp. 867–869, 2023.

[3] L. Chettri and R. Bera, "A comprehensive survey on internet of things (iot) toward 5g wireless systems," *IEEE Internet of Things Journal*, vol. 7, no. 1, pp. 16–32, 2020.

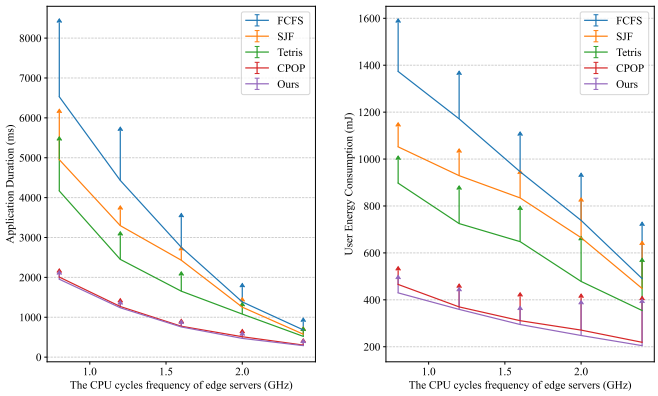
[4] Q. Wang, S. Zou, Y. Sun, M. Liwang, X. Wang, and W. Ni, "Toward intelligent and adaptive task scheduling for 6g: An intent-driven framework," *IEEE Transactions on Cognitive Communications and Networking*, vol. 10, no. 5, pp. 1975–1988, 2024.

[5] Y. Peng, B. Shi, T. Jiang, X. Tu, D. Xu, and K. Hua, "A survey on in-vehicle time-sensitive networking," *IEEE Internet of Things Journal*, vol. 10, no. 16, pp. 14 375–14 396, 2023.

[6] L. Wang, W. Su, F. Song, and I. You, "Integrating smart computility for subflow orchestration in remote virtual services," *IEEE Journal of*

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on November 05,2025 at 08:36:05 UTC from IEEE Xplore. Restrictions apply.
© 2025 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Wireless Communications. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TWC.2025.3619100



(a) Application Duration (b) Energy Consumption

Fig. 12. The performance comparison for different computing resource configurations.

Biomedical and Health Informatics, vol. 28, no. 11, pp. 6338–6347, 2024.

[7] S. K. Sahu, M. Gharbaoui, A. L. Ruscelli, G. Cecchetti, A. Sgambelluri, and P. Castoldi, "Remote medical support in emergency scenarios: A webrtc-based solution," *IEEE Communications Magazine*, pp. 1–7, 2024.

[8] J. Feng and J. Zhao, "Resource allocation for augmented reality empowered vehicular edge metaverse," *IEEE Transactions on Communications*, pp. 1–1, 2023.

[9] L. Zhang, X. Wu, F. Wang, A. Sun, L. Cui, and J. Liu, "Edge-based video stream generation for multi-party mobile augmented reality," *IEEE Transactions on Mobile Computing*, vol. 23, no. 1, pp. 409–422, 2024.

[10] Q. Cheng, H. Shan, W. Zhuang, L. Yu, Z. Zhang, and T. Q. S. Quek, "Design and analysis of mec- and proactive caching-based 360° mobile vr video streaming," *IEEE Transactions on Multimedia*, vol. 24, pp. 1529–1544, 2022.

[11] X. Chi, H. Chen, G. Li, Z. Ni, N. Jiang, and F. Xia, "Edsp-edge: Efficient dynamic edge service entity placement for mobile virtual reality systems," *IEEE Transactions on Wireless Communications*, vol. 23, no. 4, pp. 2771–2783, 2024.

[12] L. Ye, L. Yang, Y. Xia, and X. Zhao, "A cost-driven intelligence scheduling approach for deadline-constrained iot workflow applications in cloud computing," *IEEE Internet of Things Journal*, vol. 11, no. 9, pp. 16 033–16 047, 2024.

[13] S. Duan, D. Wang, J. Ren, F. Lyu, Y. Zhang, H. Wu, and X. Shen, "Distributed artificial intelligence empowered by end-edge-cloud computing: A survey," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 1, pp. 591–624, 2023.

[14] M. Verucchi, M. Theile, M. Caccamo, and M. Bertogna, "Latency-aware generation of single-rate dags from multi-rate task sets," in *2020 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*. IEEE, 2020, pp. 226–238.

[15] C. Shan, Y. Xia, Y. Zhan, and J. Zhang, "Kubeadaptor: A docking framework for workflow containerization on kubernetes," *Future Generation Computer Systems*, vol. 148, pp. 584–599, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167739X2300242X

[16] A. H. Sifat, X. Deng, B. Bharmal, S. Wang, S. Huang, J. Huang, C. Jung, H. Zeng, and R. Williams, "A safety-performance metric enabling computational awareness in autonomous robots," *IEEE Robotics and Automation Letters*, vol. 8, no. 9, pp. 5727–5734, 2023.

[17] A. Hamann, D. Dasari, F. Wurst, I. Sanudo, N. Capodieci, and P. Burgio, "Waters industrial challenge 2019 final," 2019.

[18] C. Somarakis, Y. Ghaedsharaf, and N. Motee, "Risk of collision and detachment in vehicle platooning: Time-delay–induced limitations and tradeoffs," *IEEE Transactions on automatic control*, vol. 65, no. 8, pp. 3544–3559, 2019.

[19] S. S. Narasimhan, S. Bhat, and S. P. Chinchali, "Safe networked robotics with probabilistic verification," *IEEE Robotics and Automation Letters*, vol. 9, no. 3, pp. 2917–2924, 2023.

[20] M.-M. Aseman-Manzar, S. Karimian-Aliabadi, R. Entezari-Maleki, B. Egger, and A. Movaghar, "Cost-aware resource recommendation for dag-based big data workflows: An apache spark case study," *IEEE Transactions on Services Computing*, vol. 16, no. 3, pp. 1726–1737, 2023.

[21] S. Baruah, "An ilp representation of a dag scheduling problem," *Real-Time Systems*, vol. 58, no. 1, pp. 85–102, 2022.

[22] J. Singh, S. Betha, B. Mangipudi, and N. Auluck, "Contention aware energy efficient scheduling on heterogeneous multiprocessors," *IEEE Transactions on Parallel and Distributed Systems*, vol. 26, no. 5, pp. 1251–1264, 2015.

[23] S. Tang, C. Yu, J. Sun, B.-S. Lee, T. Zhang, Z. Xu, and H. Wu, "Easypdp: An efficient parallel dynamic programming runtime system for computational biology," *IEEE Transactions on Parallel and Distributed Systems*, vol. 23, no. 5, pp. 862–872, 2012.

[24] J. Niu, C. Liu, Y. Gao, and M. Qiu, "Energy efficient task assignment with guaranteed probability satisfying timing constraints for embedded systems," *IEEE Transactions on Parallel and Distributed Systems*, vol. 25, no. 8, pp. 2043–2052, 2014.

[25] B. Hu, Z. Yan, and M. Zhao, "Workload-aware scheduling of multiple-criticality real-time applications in vehicular edge computing system," *IEEE Transactions on Industrial Informatics*, vol. 19, no. 10, pp. 10 091–10 101, 2023.

[26] M. I. Khaleel, "Failure-aware resource provisioning for hybrid computation offloading in cloud-assisted edge computing using gravity reference approach," *Swarm and Evolutionary Computation*, vol. 91, p. 101704, 2024.

[27] L.-C. Canon and E. Jeannot, "Correlation-aware heuristics for evaluating the distribution of the longest path length of a dag with random weights," *IEEE Transactions on Parallel and Distributed Systems*, vol. 27, no. 11, pp. 3158–3171, 2016.

[28] N. Kumar and D. P. Vidyarthi, "A novel hybrid pso–ga meta-heuristic for scheduling of dag with communication on multiprocessor systems," *Engineering with Computers*, vol. 32, pp. 35–47, 2016.

[29] M. H. Shirvani, "A hybrid meta-heuristic algorithm for scientific workflow scheduling in heterogeneous distributed computing systems," *Engineering Applications of Artificial Intelligence*, vol. 90, p. 103501, 2020.

[30] X. Zhang, D. Zhang, W. Zheng, and J. Chen, "An enhanced priority-based scheduling heuristic for dag applications with temporal unpredictability in task execution and data transmission," *Future Generation Computer Systems*, vol. 100, pp. 428–439, 2019.

[31] J. Yang, F. Lin, C. Chakraborty, K. Yu, Z. Guo, A.-T. Nguyen, and J. J. P. C. Rodrigues, "A parallel intelligence-driven resource scheduling scheme for digital twins-based intelligent vehicular systems," *IEEE Transactions on Intelligent Vehicles*, vol. 8, no. 4, pp. 2770–2785, 2023.

[32] L. Liu, J. Feng, X. Mu, Q. Pei, D. Lan, and M. Xiao, "Asynchronous deep reinforcement learning for collaborative task computing and on-demand resource allocation in vehicular edge computing," *IEEE Transactions on Intelligent Transportation Systems*, vol. 24, no. 12, pp. 15 513–15 526, 2023.

[33] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, "Resource management with deep reinforcement learning," in *Proceedings of the 15th ACM Workshop on Hot Topics in Networks*, ser. HotNets '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 50–56.

[34] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. A. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.

[35] X. Song, L. Liu, J. Fu, X. Zhang, J. Feng, and Q. Pei, "A reinforcement learning-based dag tasks scheduling in edge-cloud collaboration systems," in *GLOBECOM 2023 - 2023 IEEE Global Communications Conference*, 2023, pp. 1771–1776.

[36] M. Goudarzi, H. Wu, M. Palaniswami, and R. Buyya, "An application placement technique for concurrent iot applications in edge and fog computing environments," *IEEE Transactions on Mobile Computing*, vol. 20, no. 4, pp. 1298–1311, 2021.

[37] H. Xiao, C. Xu, Y. Ma, S. Yang, L. Zhong, and G.-M. Muntean, "Edge intelligence: A computational task offloading scheme for dependent iot application," *IEEE Transactions on Wireless Communications*, vol. 21, no. 9, pp. 7222–7237, 2022.

[38] G. P. Koslovski, K. Pereira, and P. R. Albuquerque, "Dag-based workflows scheduling using actor–critic deep reinforcement learning," *Future Generation Computer Systems*, vol. 150, pp. 354–363, 2024.

[39] H. Liu, W. Huang, D. I. Kim, S. Sun, Y. Zeng, and S. Feng, "Towards efficient task offloading with dependency guarantees in vehicular edge networks through distributed deep reinforcement learning," *IEEE Transactions on Vehicular Technology*, pp. 1–16, 2024.

[40] M. Goudarzi, M. Palaniswami, and R. Buyya, "A distributed deep reinforcement learning technique for application placement in edge and

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on November 05,2025 at 08:36:05 UTC from IEEE Xplore. Restrictions apply.
© 2025 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Wireless Communications. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TWC.2025.3619100

fog computing environments," *IEEE Transactions on Mobile Computing*, vol. 22, no. 5, pp. 2491–2505, 2023.

[41] S. Liu, Y. Yu, X. Lian, Y. Feng, C. She, P. L. Yeoh, L. Guo, B. Vucetic, and Y. Li, "Dependent task scheduling and offloading for minimizing deadline violation ratio in mobile edge computing networks," *IEEE Journal on Selected Areas in Communications*, vol. 41, no. 2, pp. 538–554, 2023.

[42] M. Peng, K. Zhang, J. Jiang, J. Wang, and W. Wang, "Energy-efficient resource assignment and power allocation in heterogeneous cloud radio access networks," *IEEE Transactions on Vehicular Technology*, vol. 64, no. 11, pp. 5275–5287, 2015.

[43] J. Van Leeuwen, *Handbook of theoretical computer science (vol. A) algorithms and complexity*. Mit Press, 1991.

[44] J. D. Ullman, "Np-complete scheduling problems," *Journal of Computer and System sciences*, vol. 10, no. 3, pp. 384–393, 1975.

[45] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *IEEE Transactions on Neural Networks*, vol. 20, no. 1, pp. 61–80, 2009.

[46] Y. Luo, V. Thost, and L. Shi, "Transformers over directed acyclic graphs," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[47] M. Towers, A. Kwiatkowski, J. Terry, J. U. Balis, G. De Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, A. KG *et al.*, "Gymnasium: A standard interface for reinforcement learning environments," *arXiv preprint arXiv:2407.17032*, 2024.

[48] Alibaba cluster trace program. [Online]. Available: https://github.com/alibaba/clusterdata/tree/master/cluster-trace-v2018

[49] M. Fey and J. E. Lenssen, "Fast graph representation learning with PyTorch Geometric," in *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

[50] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE transactions on parallel and distributed systems*, vol. 13, no. 3, pp. 260–274, 2002.

[51] R. Grandl, G. Ananthanarayanan, S. Kandula, S. Rao, and A. Akella, "Multi-resource packing for cluster schedulers," *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 4, pp. 455–466, 2014.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on November 05,2025 at 08:36:05 UTC from IEEE Xplore. Restrictions apply.
© 2025 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted,
but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.